

KANDIDATNUMMER(E)/NAVN:

Ottersbo, Elias Alexander Wiklund (10072)
Evensen, Sigurd (10076)

DATO:

26/5 - 2026

FAGKODE:

IDATT2003

STUDIUM:

BIDATA - TRONDHEIM

ANT SIDER/BILAG:

22 /

FAGLÆRER(E) :

- Atle Olsø
- Majid Rouhani

TITTEL :

IDATT2003 Programmering 2 – Mappevurdering 2026 Rapport

SAMMENDRAG:

Denne rapporten dokumenterer design og utvikling av Millions, et aksjemarkedssimuleringsspill utviklet som en del av mappeoppgaven i IDATT2003 Programmering 2. Applikasjonen lar brukeren søke etter aksjer, kjøpe og selge andeler, følge utviklingen i porteføljen over tid, og lagre og laste inn spilltilstand. Løsningen er implementert som en desktop-applikasjon med grafisk brukergrensesnitt bygget i JavaFX, mens kjernefunksjonaliteten er organisert i egne modell-, kontroller- og infrastrukturelag.

Prosjektet legger vekt på objektorientert design, lagdelt arkitektur, robust validering, persistens med SQLite og automatisert testing med JUnit. Rapporten beskriver kravgrunnlaget, teoretisk fundament, utviklingsprosess, teknisk løsning, teststrategi og en drøfting av både produktet og arbeidsprosessen.

Denne oppgaven er en besvarelse utført av student(er) ved NTNU.

Deklarasjon om KI-hjelpemidler

Har det i utarbeidingen av denne rapporten blitt anvendt KI-baserte hjelpemidler?

☐ Nei

☒ Ja

Hvis *ja*: spesifiser type av verktøy og bruksområde under.

Tekst

☐

Stavekontroll. Er deler av teksten kontrollert av:
Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinkai eller lignende verktøy?

☐

Tekstgenerering. Er deler av teksten generert av:
ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr eller lignende verktøy?

☐

Skriveassistanse. Er en eller flere av ideene eller fremgangsmåtene i oppgaven foreslått av:
ChatGPT, Google Bard, Bing chat, YouChat eller lignende verktøy?

Hvis *ja* til anvendelse av et tekstverktøy - spesifiser bruken her:

Kode og algoritmer

☐

Programmeringsassistanse. Er deler av koden/algoritmene som i) fremtrer direkte i rapporten eller ii) har blitt anvendt for produksjon av resultater slik som figurer, tabeller eller tallverdier blitt generert av: *GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard* eller lignende verktøy?

Hvis *ja* til anvendelse av et programmeringsverktøy - spesifiser bruken her:

Bilder og figurer

☐

Bildegenerering. Er ett eller flere av bildene/figurene i rapporten blitt generert av: *Midjourney, Jasper, WriteSonic, Stability AI, Dall-E* eller lignende verktøy?

Hvis *ja* til anvendelse av et bildeverktøy - spesifiser bruken her:

Andre KI-verktøy

☒

Andre KI-verktøy. har andre typer av verktøy blitt anvendt? Hvis *ja* spesifiser bruken her:

Se kapittel 3.3.

☒

X

Jeg er kjent med NTNUs regelverk: *Det er ikke tillatt å generere besvarelse ved hjelp av kunstig intelligens og levere den helt eller delvis som egen besvarelse.* Jeg har derfor redegjort for all anvendelse av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet

(KOMMER SENERE)

Underskrift/Dato/Sted

INNHold

Innholdsfortegnelse

1	Introduksjon.....	1
1.1	Bakgrunn	1
1.2	Kravspesifikasjon	2
1.3	Avgrensninger.....	3
1.4	Begreper/Ordliste	4
2	Teori.....	5
2.1	Single Responsibility Principle - SRP	5
2.2	Lav kobling og høy samhörighet	5
2.3	Separation of concerns og lagdelt arkitektur	5
2.4	Enkapsulering og validering.....	5
2.5	Immutabilitet og defensiv programmering.....	5
2.6	Persistens og repository-design.....	6
2.7	Automatisert testing.....	6
2.8	Don't Repeat Yourself - DRY	6
2.9	Model-View-Controller - MVC	6
3	Metode.....	7
3.1	Utviklingsprosess	7
3.2	Verktøy	8
3.3	Bruk av KI verktøy.....	8
4	Resultat.....	9
4.1	Teknisk Design	9
4.2	Implementasjon	9
4.3	Testing	10
4.4	Utrulling til sluttbruker (deployment)	11
5	Drøfting.....	12
5.1	Drøfting av løsning/design.....	12
5.2	Drøfting av prosess	13
5.3	Drøfting av bruken av KI-verktøy	13
6	Konklusjon - erfaring	14

Figurliste

Figur 1 Use Case diagram.....	2
Figur 2 Klassediagram som viser.....	Feil! Bokmerke er ikke definert.

Tabelliste

Tabell 1 Begreper og ordliste	4
-------------------------------------	---

1 INTRODUKSJON

1.1 *Bakgrunn*

Dette prosjektet omhandler utviklingen av et aksjemarkedssimuleringsspill kalt Millions, utviklet som en del av mappeoppgaven i emnet IDATT2003 Programmering 2. Formålet med applikasjonen er å simulere handel i et forenklet aksjemarked, der brukeren kan kjøpe og selge aksjer, følge utviklingen i egen portefølje, se transaksjonshistorikk og observere hvordan aksjepriser endrer seg over tid.

I motsetning til en mindre, rent algoritmisk oppgave, krevde dette prosjektet utvikling av en helhetlig programvareløsning med både grafisk brukergrensesnitt og en intern arkitektur som håndterer domenelogikk, validering, lagring og administrasjon av tilstand. Prosjektet kombinerer dermed praktisk programvareutvikling med objektorientert design, brukerinteraksjon og fokus på vedlikeholdbar kode.

Den endelige løsningen er implementert som en Java-basert desktop-applikasjon med JavaFX som brukergrensesnitt. Programmet laster aksjedata fra CSV-filer, simulerer ukentlige endringer i markedet, lar brukeren gjennomføre handler, og lagrer spilltilstanden ved hjelp av en SQLite-basert persistensløsning. På denne måten går prosjektet utover en enkel prototype og fremstår som et mer helhetlig programvareprodukt.

1.2 Kravspesifikasjon

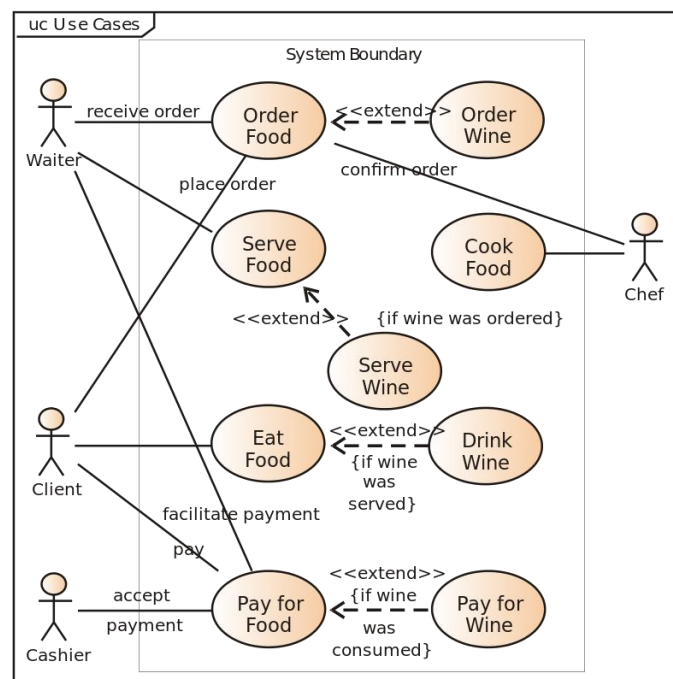
Oppgaven krever utvikling av en fungerende programvareløsning i Java som demonstrerer god objektorientert struktur, kodekvalitet og bruk av relevante programvaretekniske prinsipper. Basert på implementasjonen i repositoryet er følgende funksjonelle krav sentrale i løsningen:

- systemet skal la brukeren starte et nytt spill med valgt navn og startkapital
- brukeren skal kunne se tilgjengelige aksjer i markedet
- brukeren skal kunne søke etter aksjer basert på symbol eller selskapsnavn
- brukeren skal kunne kjøpe aksjer
- brukeren skal kunne selge aksjer som eies
- systemet skal holde oversikt over brukerens portefølje
- systemet skal lagre historikk over gjennomførte transaksjoner
- systemet skal simulere tidsprogresjon i form av uker
- aksjepriser skal endres over tid
- brukeren skal kunne lagre og laste inn spilltilstand
- løsningen skal presenteres gjennom et grafisk brukergrensesnitt

I tillegg til de funksjonelle kravene finnes det flere ikke-funksjonelle krav som er viktige for kvaliteten på løsningen:

- løsningen bør være modulær og vedlikeholdbar
- implementasjonen skal demonstrere objektorienterte designprinsipper
- koden skal inneholde tydelig validering og feilbehandling
- persistensløsningen skal være strukturert og robust
- testing skal brukes for å verifisere sentrale deler av systemet
- applikasjonen skal være forståelig og brukervennlig

Et viktig kvalitetskriterium for en sterk besvarelse er ikke at designvalgene kan begrunnes og knyttes til gode programvaretekniske prinsipper. Derfor drøfter rapporten løsningen opp mot blant annet kobling, samhörighet, ansvarsdeling, lagdelt arkitektur, validering og persistensdesign.



Figur 1 Use Case diagram

1.3 Avgrensninger

Oppgaven innebar at naturlige avgrensninger ble gjort; både som følge av oppgavens omfang, og valg som er gjort under utviklingen.

For det første er systemet en simulering, ikke en reell handelsplattform. Aksjeprisene genereres og oppdateres internt i systemet, i stedet for å hentes fra eksterne markedsdata eller sanntids-API-er. Applikasjonen legger derfor større vekt på modellering, struktur og brukerinteraksjon enn på finansiell realisme.

For det andre er persistensløsningen lokal. Lagrede spill lagres i SQLite på brukerens egen maskin, og systemet støtter verken skybasert lagring, autentisering eller flerbrukerfunksjonalitet.

For det tredje er brukergrensesnittet, selv om det er grafisk og mer brukervennlig enn en konsollapplikasjon, fortsatt begrenset til det som er hensiktsmessig innenfor rammene av et emneprosjekt. Funksjoner som avanserte grafer, pålogging, nettverksstøtte eller mer realistiske markedsmekanismer er bevisst utelatt for å kunne fokusere på kjernefunksjonalitet og kodekvalitet.

Til slutt er enkelte implementasjonsvalg gjort som følge av spesifikasjonen ovenfor ideell domenemodellering. Ett eksempel er bruk av **BigDecimal** for aksjeantall, noe som kommenteres i koden som et litt unaturlig valg for en realistisk løsning, men som likevel er fulgt for å samsvare med oppgavens premisser.

1.4 Begreper/Ordliste

Begrep (Norsk)	Begrep (Engelsk)	Betyding/beskrivelse
Aksje	Stock	Et omsettelig finansielt objekt med symbol, selskapsnavn og prishistorikk.
Aksjepost	Share / Share holding	En beholdning av et gitt antall aksjer i ett selskap, med tilhørende kjøpspris.
Portefølje	Portfolio	Samlingen av aksjer som brukeren eier.
Børs	Exchange	Markedet hvor aksjene er registrert og handles.
Transaksjon	Transaction	En gjennomført kjøps- eller salgsoperasjon.
Kjøp	Purchase	En transaksjon hvor brukeren kjøper aksjer og betaler pris pluss gebyr.
Salg	Sale	En transaksjon hvor brukeren selger aksjer og mottar netto beløp etter gebyr og eventuell skatt.
Spilltilstand	Game state	Den samlede tilstanden til spillet, inkludert børs og spillerdata.
Persistens	Persistence	Mekanismen for å lagre og laste spilldata.
Markedsbevegelse	Market movement	Ukentlig prisendring på aksjer i simuleringen.
Startkapital	Starting capital	Beløpet brukeren starter spillet med.
Transaksjonsarkiv	Transaction archive	Historikk over gjennomførte handler.

Tabell 1 Begreper og ordliste

2 TEORI

2.1 *Single Responsibility Principle*

Single Responsibility Principle innebærer at en klasse bør ha ett hovedansvar, eller én primær grunn til å endres. Dette betyr ikke at en klasse bare kan inneholde en metode, men at metodene bør støtte opp om ett klart formål.

Dette prinsippet er særlig viktig i systemer som kombinerer flere hensyn, som domenelogikk, persistens og brukergrensesnitt. Hvis disse blandes for mye sammen, blir løsningen vanskeligere å vedlikeholde, utvide og teste. Klasser med tydelig ansvar gjør systemet enklere å forstå og reduserer risikoen for at endringer i én del skaper feil i en annen.

2.2 *Lav kobling og høy samhörighet*

Lav kobling betyr at ulike deler av systemet skal være så lite avhengige av hverandre som mulig. Høy samhörighet betyr at elementene innad i en klasse eller modul bør høre naturlig sammen og støtte samme ansvar.

Disse prinsippene er viktige for vedlikeholdbarhet og testbarhet. En klasse med høy samhörighet er lettere å forstå fordi den har et tydelig formål. Et system med lav kobling er enklere å videreutvikle fordi endringer i en klasse i mindre grad tvinger frem endringer andre steder.

2.3 *Separation of concerns og lagdelt arkitektur*

Separation of Concerns handler om å dele systemet inn i deler som håndterer ulike typer ansvar, som for eksempel domenelogikk, presentasjon og lagring. Lagdelt arkitektur er en praktisk måte å realisere dette på.

I en lagdelt løsning håndterer domenemodellen forretningsreglene, kontrollere koordinerer operasjoner, visningslaget står for presentasjon, og infrastrukturet tar seg av ting som filinnlasting og databasekommunikasjon. Denne oppdelingen gjør løsningen enklere å teste og gjør det mulig å videreutvikle én del uten å måtte endre hele systemet.

2.4 *Enkapsulering og validering*

Enkapsulering innebærer at intern tilstand skal beskyttes og kun eksponeres gjennom kontrollerte grensesnitt. Tett knyttet til dette er validering: domeneobjekter bør hindre at ugyldig tilstand oppstår allerede ved opprettelse eller endring.

En robust løsning sørger for at ugyldige verdier, som tomme navn, negative beløp, ugyldige priser eller ulovlige antall, avvises tidlig. Dette reduserer behovet for defensiv kode andre steder i systemet og styrker korrektheten i løsningen som helhet.

2.5 *Immutabilitet og defensiv programmering*

Immutabilitet kan gjøre objekter mer pålitelige og lettere å resonnerer om. Selv når full immutabilitet ikke er praktisk, kan defensiv programmering brukes for å beskytte systemets interne tilstand. Et typisk eksempel er å returnere umodifiserbare samlinger i stedet for direkte referanser til interne lister.

Dette er spesielt relevant i modell- og repositoryklasser, der eksponering av mutable datastrukturer ellers kan føre til at andre deler av systemet utilsiktet bryter med etablerte invariants.

2.6 Persistens og repository-design

Persistens handler om å lagre applikasjonens tilstand slik at den kan gjenopprettes senere. I programvareutvikling er det vanlig å kapsle dette inn bak en repository-abstraksjon, slik at lagringsteknologi ikke blir blandet inn i forretningslogikken.

Repository-basert persistens gir bedre modularitet fordi det isolerer valg av lagringsteknologi fra resten av applikasjonen.

2.7 Automatisert testing

Automatisert testing er en sentral praksis for å validere oppførsel og redusere risikoen for regresjoner. Enhetstester er særlig nyttige når systemet inneholder forretningsregler, validering og beregninger som må forbli korrekte selv om koden videreutvikles.

2.8 Don't Repeat Yourself

Don't repeat yourself, er et prinsipp som er ganske selvforklarende: ikke gjenta deg. Kode som kan gjenbrukes flere steder, skal bli gjort om til funksjoner som du kan kalle på istedenfor å skrive samme kode på nytt flere steder. Dette gjør vedlikehold lettere og du sparer mye tid.

2.9 Model-View-Controller

Model-view-controller er en design arkitektur for applikasjoner, hvor applikasjonen blir delt inn i tre deler:

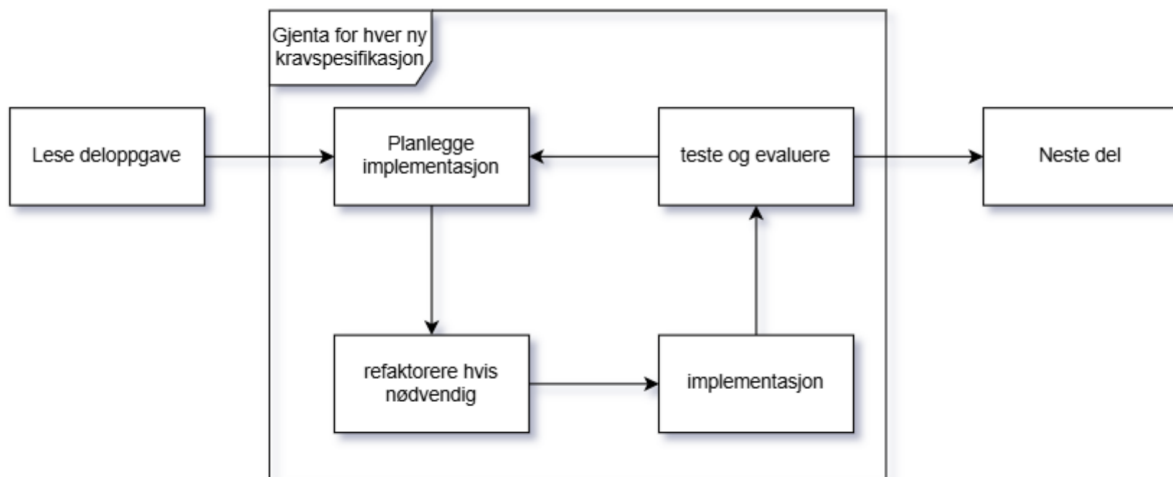
- Model: data og logikk
- View: brukergrensesnitt, altså alt øyet kan se
- Controller: sammenkoblingen mellom view og controller

Model-view-controller er ønskelig da det støtter separation of concerns og low coupling.

3 METODE

3.1 Utviklingsprosess

Ettersom at prosjektet ble gjennomført som en samarbeidsbasert mappeoppgave, stilte utviklingsprosessen krav både til teknisk struktur og koordinering mellom oss. Mellom oss ble vi enige om å ta i bruk en iterativ og inkrementell arbeidsform i stedet for å implementere hele systemet i én sammenhengende fase. Løsningen ble derfor utviklet stegvis, der grunnleggende funksjonalitet først ble etablert og deretter utvidet og forbedret. Nedenfor er en visualisering av hvordan vi jobbet gjennom hver del av prosjektet.



En tidlig prioritet var å identifisere de viktigste domenebegrepene og etablere en arkitektur som kunne støtte videre vekst. Dette innebar blant annet å modellere spilleren, aksjene, porteføljen, transaksjoner og børsen, samtidig som det måtte tas stilling til hvordan brukergrensesnitt, domenelogikk og persistens skulle samhandle. Når denne grunnstrukturen var på plass, kunne videre utvikling skje i gjentatte runder med planlegging, implementasjon, testing og forbedring.

Den iterative prosessen gjorde det også mulig å tilpasse løsningen underveis. Enkelte designvalg blir først virkelig forståelige når systemet begynner å ta form, og det var derfor naturlig å justere struktur og ansvar etter hvert som prosjektet utviklet seg.

Gjennom hele arbeidsprosessen ble branches brukt til å holde styr på utviklingen av prosjektet. Main branch er kun fungerende kode som i teorien er produksjonsklart. Dev branch ble brukt til code review før merge til main, mens spesifikke branches ble brukt for utvikling av features. Dersom noe skulle stoppe å fungere, er det dermed lett å se hvilke PR's som har blitt gjort inn i dev, og derav revert eller fikse problemet. Siden vi ikke pusher rett til main vil kode på main alltid fungere. Dette sørget vi for med main branch protection.

3.2 Verktøy

Verktøy	Versjon	Hensikt
Java Development Kit (JDK)	25	Programmeringsspråk og runtime environment
Apache Maven	POM 4.0.0	Bygging av prosjekt, avhengigheter og testkjøring
Maven Compiler Plugin	3.14.1	Kompilering mot Java 25
Maven Surefire Plugin	3.5.4	Kjøre JUnit-tester
JavaFX Controls	25.0.1	Grafisk brukergrensesnitt
SQLite JDBC	3.45.1.0	Databasekommunikasjon for persistens
JUnit Jupiter	6.0.1	Automatisert testing
IntelliJ IDEA	<i>[fyll inn ved behov]</i>	Utviklingsmiljø
Git	2.38.0	Versjonskontroll
GitHub	github.com	Lagring og samarbeid
Draw.io / PlantUML	<i>[fyll inn ved behov]</i>	Diagrammer og visualisering

3.3 Bruk av KI verktøy

Ingen kode i prosjektet er skrevet av kun KI, men KI har blitt benyttet til å assistere oss i utviklingen. Det vil si; inline forslag og autocomplete fra copilot, generering av enkle kommentarer og rettskriving. En del av testene har blitt skrevet av KI, men vi har manuelt sjekket de i code review.

4 RESULTAT

4.1 Teknisk Design

Applikasjonen er en frittstående desktop-applikasjon skrevet i Java 25 med JavaFX som grafisk rammeverk. Det er ingen server, alt kjører lokalt på brukerens maskin. Denne arkitekturen ble valgt fordi kravspesifikasjonen beskriver et enkeltbruker-spill som skal kunne kjøres offline, og fordi en desktop-applikasjon eliminerer avhengigheter til ekstern infrastruktur og gjør utrulling enkelt.

Spilltilstanden lagres i en lokal SQLite-database. SQLite ble valgt fordi det ikke krever en separat databaseserver; filen lagres direkte på disk og biblioteket `sqlite-jdbc` gjør integrasjonen enkel fra Java. Dette er tilstrekkelig for et enkeltbruker-spill der datavolum er lite.

Kodebasen er delt i fire lag inspirert av MVC og Separation of Concerns:

- **model**: domeneobjekter uten avhengigheter til UI eller persistens
- **controller**: forretningslogikk og tilstandshåndtering som medierer mellom **view** og **model**
- **view**: JavaFX-komponenter som kun leser fra og kaller kontrollere, ingen logikk her
- **infrastructure**: persistens (SQLite), filinnlasting (CSV) og unntak

Denne lagdelingen gjør det mulig å teste domenelogikk og forretningslogikk uavhengig av brukergrensesnittet, og å bytte ut infrastrukturlag uten å berøre resten av kodebasen.

Aksjeuniverset (S&P 500-utvalg) lastes inn fra en CSV-fil ved oppstart via **StockCsvLoader**. Kursutvikling simuleres ved at **Exchange.advance()** oppdaterer alle aksjekurser med en tilfeldig endring på inntil $\pm 5\%$ per uke.

4.2 Implementasjon

4.2.1 Modellklasser

Stock representerer en børsnotert aksje med ticker-symbol, selskapsnavn og fullstendig prishistorikk. Gjeldende kurs er alltid siste innslag i listen. **Share** representerer et eierskap i en aksje, med beholdning og innkjøpspris.

Player holder på navn, startkapital, kontantbeholdning, **Portfolio** og **TransactionArchive**. Klassen beregner nettoverdi og avleder spillerstatus (**NOVICE**, **INVESTOR**, **SPECULATOR**) basert på antall handelsuker og nettoverdivest.

Portfolio administrerer spillerens **Share**-objekter med støtte for innlegging, fjerning og oppslag på aksjesymbol.

Transaction er en abstrakt basisklasse for transaksjoner som håndhever at en transaksjon kun kan committes en gang. **Purchase** og **Sale** er konkrete underklasser for henholdsvis kjøp og salg. Finansielle beregninger delegeres til **PurchaseCalculator** og **SaleCalculator**.

Exchange representerer børsen. Den administrerer noterte aksjer, håndterer kjøp/salg og rykker simuleringen frem én uke via **advance()**, som oppdaterer alle aksjekurser med en tilfeldig endring på inntil $\pm 5\%$.

GameState binder **Exchange**, **Player** og **NetWorthSnapshot**-historikk til én samlet tilstand for lagring og overføring.

4.2.2 Kontrollere

ExchangeController er hovedkontrolleren mellom view og modell. Den eksponerer **buy()**, **sell()** og **advance()**, og holder en løpende nettoverdi-historikk. Lagring av spilltilstand skjer asynkront via **saveGame()**.

ProfileController henter spillerstatistikk (beste/verste handler, total kjøpt/solgt, nettoverdi-historikk) og delegerer lagring til **ExchangeController**.

LoadGameController håndterer innlasting av lagrede spill og konstruerer et nytt **ExchangeController**-objekt fra valgt **GameState**.

4.2.3 Grensesnitt

StartPage er landingssiden der spilleren angir navn og startkapital eller laster et lagret spill.

DashboardPage er applikasjonens spille side, sammensatt av en header med spillerinfo og uke-knapp, et venstre panel med aksjeoversikt og søk, og et høyre panel med beholdning og transaksjonshistorikk.

TradeStockDialog er handelsdialogen for kjøp og salg. Den viser prishistorikk som graf og presenterer en kvittering etter gjennomført handel.

ProfilePage viser statistikk og en graf over nettoverdiutviklingen.

4.2.4 Infrastruktur

StockCsvLoader leser aksjedata fra CSV-fil (symbol, navn, kurs) og brukes ved oppstart til å laste inn aksjeuniverset.

SqliteGameRepository persisterer hele spilltilstanden i en SQLite-database, inkludert prishistorikk og transaksjonsarkiv, og brukes til lagring og innlasting av spill.

4.3 Testing

Prosjektet inneholder automatiserte tester skrevet med JUnit, og testene retter seg primært mot domenelogikk og støttekomponenter snarere enn direkte mot brukergrensesnittet.

Prosjektet inneholder tester for flere sentrale områder:

- **ExchangeTest** tester aksjeoppslag, kjøp, salg, søk og ukentlig progresjon
- **PortfolioTest** tester innlegging, fjerning, oppslag og defensiv oppførsel i porteføljen
- **TransactionTest** tester commit-logikken for kjøp og salg, inkludert feilsituasjoner
- **PurchaseCalculatorTest** og **SaleCalculatorTest** tester finansielle beregninger
- **StockTest** tester prishistorikk og avledede verdier
- **StockCsvLoaderTest** tester innlasting fra CSV, inkludert kommentarer, blanke linjer og feilformat
- **TransactionArchiveTest** tester transaksjonshistorikk
- **PlayerStatusTest** tester utvikling av spillerstatus basert på handelsaktivitet og verdiutvikling

En styrke ved testene er at de ikke bare fokuserer på vellykkede tilfeller, men også verifiserer ugyldig input og feilsituasjoner. For eksempel testes det for salg av aksjer man ikke eier.

Testene blir kjørt ved bruk av JUnit 6.0.1 og Maven Surefire 3.5.4

4.4 Utrulling til sluttbruker

Applikasjonen distribueres som en Java-basert desktop-applikasjon bygget med Maven. I praksis innebærer dette at brukeren må ha et kompatibelt Java-miljø tilgjengelig for å kunne kjøre programmet. README-filen i repositoryet viser at applikasjonen kan startes enten gjennom Maven eller direkte fra IDE, med JavaFX-applikasjonen som hovedinngang.

Rent teknisk består utrulling av å klonere prosjektet, la Maven hente nødvendige avhengigheter, og deretter starte programmet. Siden JavaFX og SQLite-avhengigheter er deklarerert i pom.xml, håndteres oppsettet automatisk av byggverktøyet. Dette forenkler kjøringen for en bruker som allerede har riktig Java-oppsett. Brukere kan også laste ned spesifikke versjoner fra releases på github istedenfor å klonere prosjektet.

5 DRØFTING

5.1 Drøfting av løsning/design

Løsningen oppfyller de sentrale kravene i oppgaven ved å tilby et fungerende aksjemarkedssimuleringsspill med grafisk brukergrensesnitt, kjøp og salg av aksjer, porteføljeoversikt, transaksjonshistorikk, ukentlig markedssimulering og lagring av spilltilstand. Samtidig går løsningen lenger enn en minimal implementasjon ved å kombinere GUI, persistens og relativt tydelig lagdeling.

En klar styrke ved løsningen er at sentrale begreper i domenet er modellert som egne klasser. **Stock**, **Share**, **Player**, **Portfolio**, **Exchange** og **Transaction** gjør kodebasen lett å lese fordi klassene i stor grad samsvarer med problemdomenet. Dette gir også et godt grunnlag for videreutvikling. For eksempel er beregninger for kjøp og salg skilt ut i **PurchaseCalculator** og **SaleCalculator**, i stedet for å ligge spredt i transaksjonslogikk eller brukergrensesnitt. Det bidrar både til høyere samhörighet og til at finansiell logikk blir enklere å teste isolert.

Løsningen viser også lavere kobling enn en mer direkte implementasjon ville gjort. Brukergrensesnittet er ikke ansvarlig for selve forretningslogikken, men kommuniserer gjennom kontrollere som **ExchangeController** og **ProfileController**. På samme måte er persistens lagt i egne infrastrukturen, blant annet gjennom **SqliteGameRepository**, i stedet for å være bygget inn i domeneklassene. Dette er et godt designvalg fordi endringer i lagring eller presentasjon i mindre grad tvinger frem endringer i resten av systemet.

Et annet positivt trekk er at flere modellklasser håndhever gyldig tilstand tidlig. Konstruktørene i blant annet **Player**, **Share** og **Stock** validerer input og avviser ugyldige verdier. I tillegg returnerer blant annet **Portfolio** og **Stock** umodifiserbare samlinger i stedet for å eksponere interne mutable datastrukturer direkte. Dette er konkrete eksempler på defensiv programmering og bidrar til en mer robust løsning.

Persistensløsningen er også et av prosjektets sterkere punkter. At spilltilstand lagres i SQLite, og ikke kun i minnet eller i en enkel tekstfil, gjør løsningen mer strukturert og mer realistisk som applikasjon. Det er særlig positivt at lagringen er delt opp i tabeller for spill, spillerdata, aksjer, prisutvikling, portefølje og transaksjoner. Det viser forståelse for at applikasjonstilstand består av flere ulike typer data med ulike relasjoner.

Det finnes likevel svakheter. Den tydeligste er at kodebasen bærer preg av pågående refaktorering. Repositoryet ser ut til å inneholde både `model`- og `domain`-varianter av enkelte deler av løsningen. Dette tyder på at arkitekturen har blitt forbedret underveis, men også at oppryddingen ikke er helt fullført. For sensor vil dette kunne trekke ned helhetsinntrykket, fordi en sluttinnlevering ideelt sett bør fremstå mer konsolidert.

Det finnes også enkelte valg som virker mer spesifikasjonsdrevne enn domenedrevne. Bruken av **BigDecimal** for aksjeantall er et godt eksempel. Dette kan forsvares dersom det følger av oppgaveteksten, men det er samtidig ikke et naturlig valg i en realistisk modell av aksjehandel. Det viser at løsningen i noen grad balanserer mellom å følge spesifikasjonen tett og å modellere domenet så naturlig som mulig.

Samlet vurdert er løsningen faglig sterk. Den har en tydelig kjerne, god funksjonell bredde og flere gode designvalg, særlig innen ansvarsdeling, validering, testbarhet og persistens. Det som først og fremst hindrer løsningen fra å fremstå enda sterkere, er ikke mangel på funksjonalitet, men at kodebasen kunne vært ryddigere og mer konsistent i den endelige arkitekturen.

5.2 Drøfting av prosess

Den iterative utviklingsprosessen var et riktig valg for dette prosjektet. Oppgaven bestod av flere ulike problemstillinger samtidig: domenemodellering, GUI, beregninger, persistens og testing. En lineær prosess ville gjort det vanskeligere å oppdage designfeil tidlig. Ved å jobbe stegvis ble det mulig å etablere en fungerende grunnstruktur først, og deretter forbedre løsningen etter hvert som forståelsen av oppgaven ble bedre.

For et gruppeprosjekt er dette særlig viktig. En iterativ prosess gjør det lettere å fordele arbeid mellom gruppemedlemmer uten at alle må vente på hverandre hele tiden. Når ansvarsområder og grensesnitt først er noenlunde etablert, kan én utvikler arbeide med brukergrensesnitt, en annen med persistens og en tredje med tester eller domenelogikk. Det gjør arbeidet mer effektivt og reduserer risikoen for at én del av prosjektet stopper opp hele utviklingen.

Samtidig viser repositoryets struktur at prosessen innebar forbedring og omorganisering underveis. Det er naturlig i programvareutvikling. Ofte blir det først tydelig etter at implementasjonen er påbegynt at enkelte ansvarsområder bør flyttes, abstraheres bedre eller organiseres annerledes. Derfor bør ikke refaktorering tolkes som tegn på svak planlegging, men snarere som en del av det å forbedre løsningen gjennom innsikt opparbeidet underveis.

Dersom noe skulle vært gjort annerledes, ville det vært å sette av mer tid til slutføring, og ikke bare implementasjon. Mer spesifikt: arkitektonisk opprydding og gjennomgang av pakkestruktur. Altså, selv om selve utviklingsmetodikken har fungert, og vært svært god, kunne sluttfasen likevel vært litt strammere.

Alt i alt har prosessen vært faglig fornuftig og godt egnet til oppgaven. Den ga rom for både progresjon og forbedring underveis. Det eneste forbedringspotensialet vårt er dermed at sluttproduktet fortsatt viser spor av utviklingshistorikken.

5.3 Drøfting av bruken av KI-verktøy

Innen programvareutvikling er KI nyttig til å f.eks bidra med effektivitet uten nødvendigvis å redusere den faglige egeninnsatsen. Samtidig har KI klare svakheter. Verktøyene kan produsere tekst og kode som virker overbevisende, men som ikke alltid stemmer med faktisk implementasjon eller oppgavetekst. Dermed, også i sammenheng med rapporter, er den mest forsvarlige bruken av KI å bruke den som skrivehjelp, *ikke* som erstatning for analyse.

6 KONKLUSJON

Prosjektet resulterte i et komplett og relativt godt strukturert aksjemarkedssimuleringsspill som demonstrerer relevant kompetanse innen objektorientert programmering, programvarearkitektur, persistens, testing og grafisk brukergrensesnittutvikling. Applikasjonen lar brukeren opprette eller laste inn et spill, søke etter og handle aksjer, følge prisutvikling over tid, holde oversikt over porteføljen og lagre fremgangen lokalt.

Fra et programvareteknisk perspektiv er de sterkeste sidene ved prosjektet den tydelige domenemodellen, separasjonen mellom brukergrensesnitt, kontrollere og infrastruktur, bruken av automatiserte tester for sentral logikk, og implementasjonen av strukturert persistens med SQLite. Disse valgene gjør løsningen mer vedlikeholdbar og mer representativ for reell applikasjonsutvikling enn en minimal kursprototype.

Dersom prosjektet skulle videreutvikles, finnes det flere naturlige forbedringsmuligheter. Arkitekturen kunne vært ytterligere konsolidert der refaktorering har etterlatt overlappende strukturer. Persistensflyten kunne vært styrket gjennom flere integrasjonstester. Brukergrensesnittet kunne også blitt utvidet med rikere visualiseringer, mer avanserte filtre og bedre porteføljeanalyse.

Til tross for disse mulighetene fremstår prosjektet allerede som et sterkt resultat. Det oppfyller ikke bare sentrale funksjonelle mål, men viser også refleksjon rundt designprinsipper, implementasjonskvalitet og utviklingsprosess. Derfor gir prosjektet et godt grunnlag for en rapport på høyt nivå og støtter argumentet om at løsningen kan vurderes som en sterk besvarelse.

REFERANSER

- [1] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Martin, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- [3] Oracle. *The Java Tutorials*.
- [4] OpenJFX Documentation.
- [5] SQLite Documentation.
- [6] Fowler, M. *Patterns of Enterprise Application Architecture*.
- [7] Wikipedia contributors. "Coupling (computer programming)."
- [8] Wikipedia contributors. "Cohesion (computer science)."
- [9] Wikipedia contributors. "Model-view-controller."

VEDLEGG

[Materiell som er utarbeidet eller innsamlet i tilknytning til rapporten, men som det ikke er naturlig eller hensiktsmessig å ta inn i hoveddelen, som feks brukerveiledning, skal tas inn som vedlegg.

Vedleggene skal være nummererte og ha en overskrift.

Har du/dere ingen vedlegg, så droppes dette kapittelet.]